

MongoDB CRUD Operations using Command Prompt

1. Database & Collection Creation

```
Current Mongosh Log ID: 69f63dc99d58a011c7abc113
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.2.7
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-05-02T22:50:24.027+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test>
(test> exit, press Ctrl+C again or Ctrl+D or type .exit)
test> show dbs
admin    40.00 KiB
config   72.00 KiB
local    40.00 KiB
test> use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> db.students.insertOne({
  |   name: "Arun",
  |   age: 22,
  |   course: "MERN Stack",
  |   status: "ongoing"
  | })
{
  acknowledged: true,
  insertedId: ObjectId('69f641889d58a011c7abc114')
}
studentDB> db.students.insertMany([
  | { name: "Kumar", age: 23, course: "MERN Stack", status: "ongoing" },
  | { name: "Divya", age: 21, course: "Python", status: "completed" },
  | { name: "Priya", age: 24, course: "Java", status: "ongoing" }
  | ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f641b09d58a011c7abc115'),
    '1': ObjectId('69f641b09d58a011c7abc116'),
    '2': ObjectId('69f641b09d58a011c7abc117')
  }
}
studentDB>
```

2. Insert Operations

```
studentDB> db.students.find().pretty()
[
  {
    _id: ObjectId('69f641889d58a011c7abc114'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f641b09d58a011c7abc115'),
    name: 'Kumar',
    age: 23,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f641b09d58a011c7abc116'),
    name: 'Divya',
    age: 21,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('69f641b09d58a011c7abc117'),
    name: 'Priya',
    age: 24,
    course: 'Java',
    status: 'ongoing'
  }
]
studentDB>
```

3. Read Operations

```
studentDB> db.students.updateOne(
|   { name: "Arun" },
|   { $set: { status: "completed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.find({ name: "Arun" })
[
  {
    _id: ObjectId('69f641889d58a011c7abc114'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  }
]
studentDB>
```

4. Update Operation

```
studentDB> db.students.deleteOne({ name: "Priya" })
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.find().pretty()
[
  {
    _id: ObjectId('69f641889d58a011c7abc114'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f641b09d58a011c7abc115'),
    name: 'Kumar',
    age: 23,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f641b09d58a011c7abc116'),
    name: 'Divya',
    age: 21,
    course: 'Python',
    status: 'completed'
  }
]
studentDB> |
```

5. Delete Operations

```
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 3 }
studentDB> db.students.find()

studentDB>
```

```
studentDB> db.students.insertMany([
|   { name: "Arun", age: 22, course: "MERN Stack", status: "completed" },
|   { name: "Kumar", age: 23, course: "MERN Stack", status: "ongoing" },
|   { name: "Divya", age: 21, course: "Python", status: "completed" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f644fb9d58a011c7abc118'),
    '1': ObjectId('69f644fb9d58a011c7abc119'),
    '2': ObjectId('69f644fb9d58a011c7abc11a')
  }
}
studentDB> db.students.find({ age: { $gt: 22 } })
[
  {
    _id: ObjectId('69f644fb9d58a011c7abc119'),
    name: 'Kumar',
    age: 23,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
studentDB> |
```

6. Query Operators

```
studentDB> db.students.find({ age: { $lt: 23 } })
[
  {
    _id: ObjectId('69f644fb9d58a011c7abc118'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f644fb9d58a011c7abc11a'),
    name: 'Divya',
    age: 21,
    course: 'Python',
    status: 'completed'
  }
]
studentDB> |
```

```
studentDB> db.students.find({
|   course: { $in: ["MERN Stack", "Python"] }
| })
[
  {
    _id: ObjectId('69f644fb9d58a011c7abc118'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f644fb9d58a011c7abc119'),
    name: 'Kumar',
    age: 23,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f644fb9d58a011c7abc11a'),
    name: 'Divya',
    age: 21,
    course: 'Python',
    status: 'completed'
  }
]
studentDB> |
```

```
studentDB> db.students.find({
|   $and: [
|     { course: "MERN Stack" },
|     { status: "completed" }
|   ]
| })
[
|   {
|     _id: ObjectId('69f644fb9d58a011c7abc118'),
|     name: 'Arun',
|     age: 22,
|     course: 'MERN Stack',
|     status: 'completed'
|   }
| ]
studentDB>
```



```
studentDB> db.students.find({
|   $or: [
|     { course: "Python" },
|     { age: 23 }
|   ]
| })
[
  {
    _id: ObjectId('69f644fb9d58a011c7abc119'),
    name: 'Kumar',
    age: 23,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f644fb9d58a011c7abc11a'),
    name: 'Divya',
    age: 21,
    course: 'Python',
    status: 'completed'
  }
]
studentDB>
```

```
studentDB> db.students.find({
|   status: { $exists: true }
| })
[
  {
    _id: ObjectId('69f644fb9d58a011c7abc118'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f644fb9d58a011c7abc119'),
    name: 'Kumar',
    age: 23,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f644fb9d58a011c7abc11a'),
    name: 'Divya',
    age: 21,
    course: 'Python',
    status: 'completed'
  }
]
studentDB>
```

7. Use Case: E-Commerce Book Store

```
Microsoft Windows [Version 10.0.20260.0240]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Flowerking>mongosh
Current Mongosh Log ID: 69f64714086823a248abc113
Connecting to:  mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:  8.2.7
Using Mongosh:  2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-05-02T22:50:24.027+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> show dbs
admin      40.00 KiB
config    88.00 KiB
local     40.00 KiB
studentDB 48.00 KiB
test> use ecommerceDB
switched to db ecommerceDB
ecommerceDB> db.books.insertMany([
|   { title: "MongoDB Basics", price: 500, stock: 10 },
|   { title: "Java Programming", price: 650, stock: 0 },
|   { title: "Python Guide", price: 400, stock: 5 }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f6475a086823a248abc114'),
    '1': ObjectId('69f6475a086823a248abc115'),
    '2': ObjectId('69f6475a086823a248abc116')
  }
}
ecommerceDB>
```

```
ecommerceDB> db.books.find({ stock: { $gt: 0 } })
[
  {
    _id: ObjectId('69f6475a086823a248abc114'),
    title: 'MongoDB Basics',
    price: 500,
    stock: 10
  },
  {
    _id: ObjectId('69f6475a086823a248abc116'),
    title: 'Python Guide',
    price: 400,
    stock: 5
  }
]
ecommerceDB> |
```

```
ecommerceDB> db.books.updateOne(  
|   { title: "Java Programming" },  
|   { $set: { stock: 8 } }  
| )  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
ecommerceDB> db.books.find({ title: "Java Programming" })  
[  
  {  
    _id: ObjectId('69f6475a086823a248abc115'),  
    title: 'Java Programming',  
    price: 650,  
    stock: 8  
  }  
]  
ecommerceDB> |
```

Conclusion:

MongoDB CRUD operations were successfully performed using mongosh.